

The Digital Ear

Qualifier Project # 1

https://youtu.be/-NZO0LaA_Zs - GUI Demo

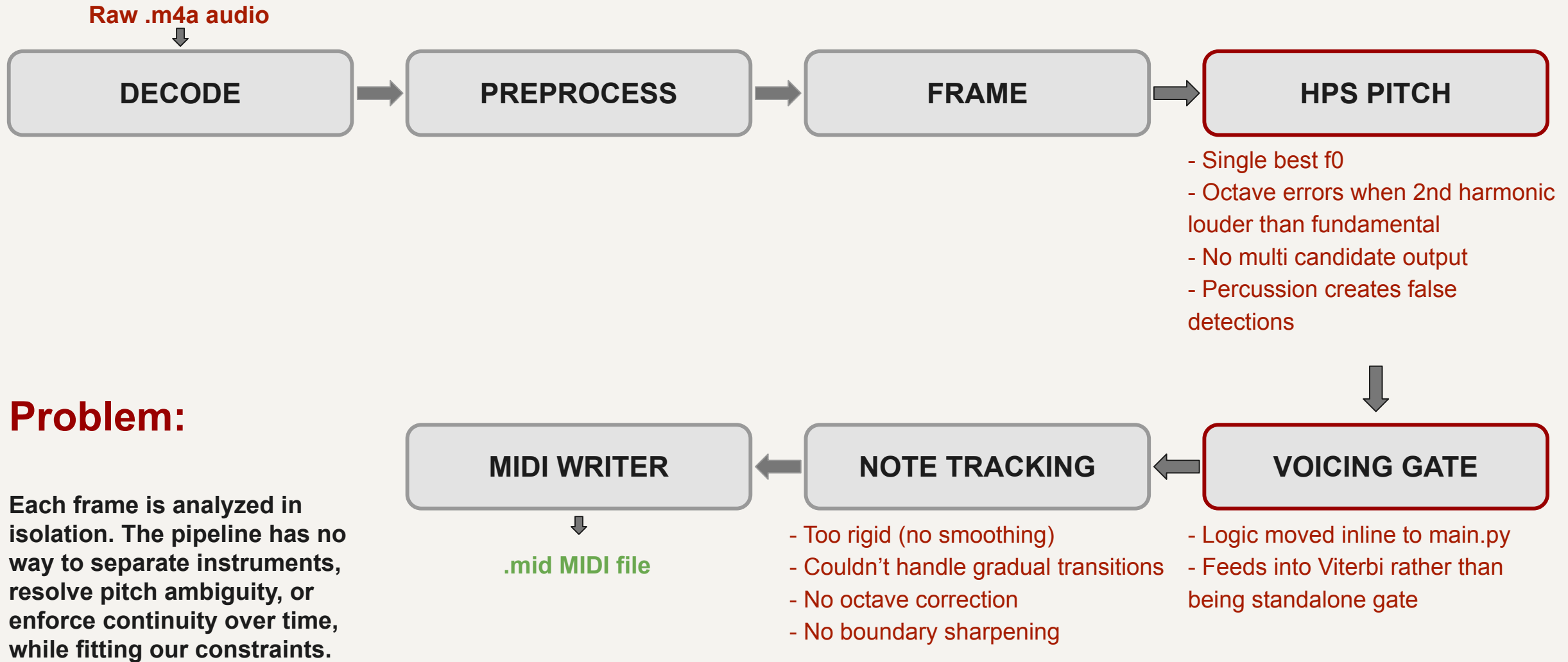
https://youtu.be/Pc_jBn6hHbY - Live Musicbox Demo

[Source Files](#)

Roadmap

- 1) First iteration
- 2) Problems and Research Pivot
- 3) New pipeline
- 4) Detailed implementation
- 5) Spectrogram
- 6) Constraints
- 7) Accuracy Evaluation
- 8) Extras

First Pipeline Overview (old)



Problem:

Each frame is analyzed in isolation. The pipeline has no way to separate instruments, resolve pitch ambiguity, or enforce continuity over time, while fitting our constraints.

Only clean signals passed.

The Research Pivot

Problem: Each frame is analyzed in isolation. The pipeline has no way to separate instruments, resolve pitch ambiguity, or enforce continuity over time, while fitting our constraints.

Problem	Paper	Solution
Percussion triggers false pitches	Fitzgerald, DAFx 2010	HPSS: separate harmonic / percussive <i>before</i> pitch detection
Weak fundamentals -> octave errors	Salamon & Gómez, 2012	MELODIA: harmonic salience in cents domain, sums 12 harmonics
Frame-to-frame pitch flickering	Bitner et al., 2015	Online HMM: smooth pitch transitions with transition costs

Tonal vs noisy sections need different tuning	Custom	Density-adaptive Viterbi: transition costs and voicing thresholds auto adjust based on causal tonality/density estimates
---	--------	--

Pitch flickering solution: **pYIN** (Mauch & Dixon, 2014) solves flickering (HMM + Viterbi), but candidates use time-domain autocorrelation and fails on percussion/polyphonic input

Current Pipeline Overview

Raw .m4a audio



DECODE

(audio_io.py)

- ffmpeg subprocess decodes .m4a, pipes raw PCM to stdout
- Mono, 44100Hz, float32, 2048 samples (or less) per block
- Never loads the file, reads from pipe

PREPROCESS

(preprocess.py)

- 3 one-pole IIR filters:
 - DC block (30Hz)
 - HPF (60Hz)
 - LPF (4000Hz)
- 3 floats of total state, carries across blocks

HPSS

(hpss.py)

- Separates harmonics (melody) from percussive (drums) via median filtering
- Time axis median: tonal content
- Frequency axis median: transients
- Wiener soft mask isolates harmonics, 414KB circular buffer, 175ms latency

FRAME

(features.py)

- Chops audio into 2048-sample Hann-windowed frames, hop 512 (75% overlap)
- One pitch estimate every ~11.6ms
- 8KB buffer, overlap ensures no missed transitions



MIDI WRITER

(midi_writer.py)

- Median smooth -> octave correct -> run-length encode -> boundary sharpen -> merge'
- Converts noisy pitch track into clean note on/off events
- Latency compensation aligning



.mid MIDI file

NOTES

(note_tracker.py)

- Median smooth -> octave correct -> run-length encode -> boundary sharpen -> merge
- Converts noisy pitch track into clean note on/off events
- Latency compensation aligning

VITERBI

(melody_extractor.py)

- Online HMM: finds smoothest pitch path through all candidates across time
- Fixed 50-frame lag (~580ms), density adaptive transition costs (custom)
- Casual octave correction via 5s running median

PITCH

(harmonic_pitch.py)

- MELODIA-style salience:
 - sums harmonic evidence in 10-cent bins (120bins/octave)
- 12 harmonics (cos^2 spreading)
 - finds fundamental
- Returns top-N candidates with conf. scores + tonality measure

MELODIA-inspired features

Salamon & Gómez, 2012

Big picture: HPS looks at one peak in one spectrum. MELODIA builds a 12-harmonic evidence map across the entire pitch range, then hands multiple hypotheses to Viterbi to resolve over time.

Cents Domain Conversion

$$B(\hat{f}) = \left\lfloor \frac{1200 \cdot \log_2 \left(\frac{\hat{f}}{55} \right)}{10} + 1 \right\rfloor.$$

Section II Eq. 5

```
# Cents-domain setup: 600 bins * 10 cents = 6000 cents (55 Hz to ~1750 Hz)
self._f_ref = 55.0
self._cent_res = 10
self._n_cent_bins = 600

self._fmin_cbin = max(0, int(np.floor(
    1200.0 * np.log2(self.fmin / self._f_ref) / self._cent_res
)))
self._fmax_cbin = min(self._n_cent_bins, int(np.ceil(
    1200.0 * np.log2(self.fmax / self._f_ref) / self._cent_res
)) + 1)
```

harmonic_pitch.py: 73-76 & 228

```
# Map to cent bins (exact float position)
cent_bins_exact = 1200.0 * np.log2(sf / self._f_ref) / self._cent_res
```

cos² Spreading

$$g(b, h, \hat{f}_i) = \begin{cases} \cos^2(\delta \cdot \frac{\pi}{2}) \cdot \alpha^{h-1} & \text{if } |\delta| \leq 1, \\ 0 & \text{if } |\delta| > 1, \end{cases}$$

Section II-B Eq. 8

Harmonic Saliency (Simplified)

$$S(b) = \sum_{h=1}^{N_h} \sum_{i=1}^I e(\hat{a}_i) \cdot g(b, h, \hat{f}_i) \cdot (\hat{a}_i)^\beta,$$

Section II Eq. 6

```
# ----- Compute saliency in cents domain -----
saliency = np.zeros(self._n_cent_bins, dtype=np.float32)
bps = 100 // self._cent_res # bins per semitone = 10
f_max_hz = self._f_ref * 2 ** (self._n_cent_bins * self._cent_res / 1200.0)
half_pi = np.pi / 2.0
offsets = np.arange(-bps, bps + 1, dtype=np.int32) # shape (21,)

for h in range(1, self.n_harmonics + 1):
    weight_h = self.alpha ** (h - 1)
    sub_freqs = peak_freqs / h

    # Filter to valid cents range
    valid = (sub_freqs >= self._f_ref) & (sub_freqs < f_max_hz)
    if not valid.any():
        continue
```

harmonic_pitch.py: 208-248

IF Refinement

$$\hat{f}_i = (k_i + \kappa(k_i)) \frac{f_s}{N},$$

$$\kappa(k_i) = \frac{N}{2\pi H} \Psi \left(\phi_l(k_i) - \phi_{l-1}(k_i) - \frac{2\pi H}{N} k_i \right),$$

Section II Eq. 2 & 3

```
# IF refinement: use phase difference for more accurate frequencies
if phase is not None and self._prev_phase is not None:
    # Expected phase advance per bin per hop
    expected_advance = 2.0 * np.pi * peak_indices * self.hop / self.zero_pad
    # Actual phase difference
    dp = phase[peak_indices] - self._prev_phase[peak_indices]
    # Deviation from expected (unwrap to [-pi, pi])
    dev = dp - expected_advance
    dev = (dev + np.pi) % (2.0 * np.pi) - np.pi
    # IF = bin_freq + deviation / (2*pi) * (sr / hop)
    if_freqs = (peak_indices * self._bin_hz) + dev / (2.0 * np.pi) * (self.sr / self.hop)
    # Use IF when it's within 2 bins of the magnitude peak (reasonable)
    reasonable = np.abs(if_freqs - peak_indices * self._bin_hz) < (2.0 * self._bin_hz)
    freqs = np.where(reasonable, if_freqs, freqs)
```

harmonic_pitch.py: 140-153

```
deltas = np.abs(target_bins - cent_bins_exact[:, None]) / bps
cos_w = np.where(deltas <= 1.0, np.cos(deltas * half_pi) ** 2, 0.0)
```

harmonic_pitch.py: 234-235

and more...

Density-adaptive Viterbi (Custom)

What and why: Standard Viterbi uses fixed costs. A piano melody, and a drum break need different tracking behavior. My solution adapts two costs: *transition (pitch jump penalty)* and *voicing (voicing↔unvoiced penalty)*, based on two causal signal estimates.

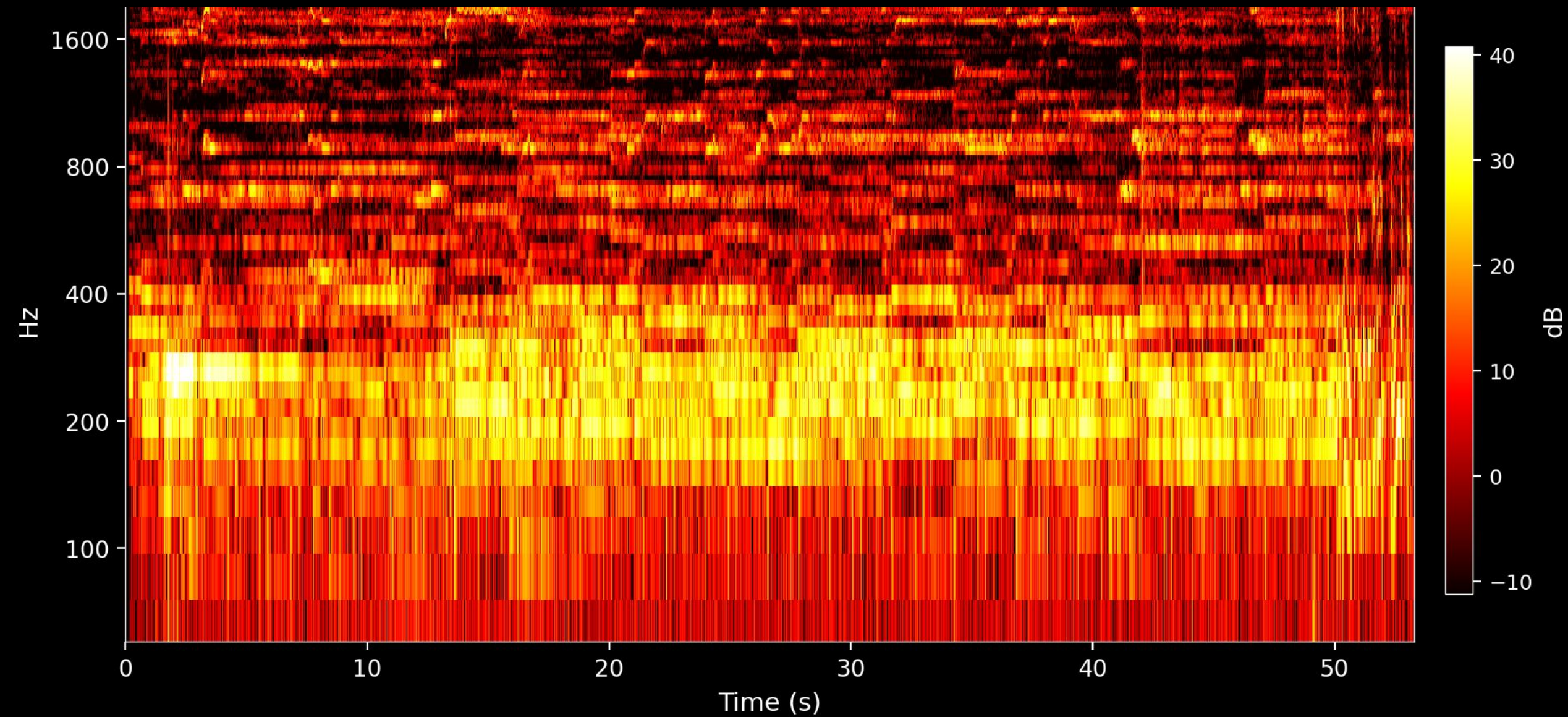
What	Adapts To	Effect	Line
<i>jump_w</i> (pitch jump cost)	tonality x density	Tonal → tight (1.2/ST), Noisy → loose (0.5/ST)	230-243
<i>voice_c</i> (voicing cost)	tonality x density	Tonal → easy to go unvoiced (4.0), Noisy → resist (7.0)	234-243
<i>emission</i> (candidate score)	density	Sparse sections → boost confidence so melody isn't lost	220-223

	Value	Constraint
Lag	50 frames= ~580 ms	Causal, fixed, no future lookahead
Memory	~23 KB constant	Ring buffer (50 entries) + 3 bounded dequeues
Lookback windows	0.5s tonality, 10s density, 5s octave	All bounded dequeues, no growth

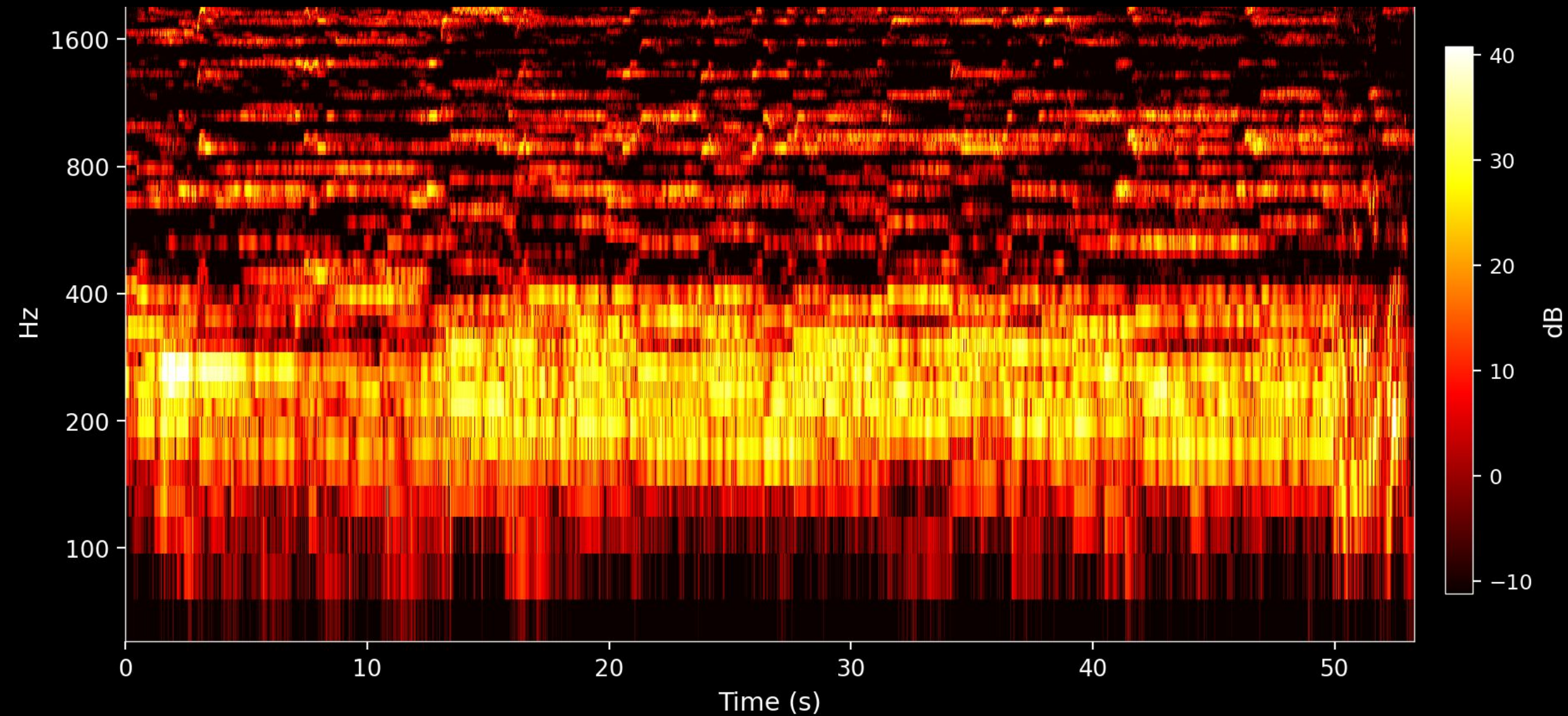
GOAL:

Pick the smoothest path through all frame candidates. Aim for stable, smooth melody contour with fewer octave jumps, false notes, and dropouts

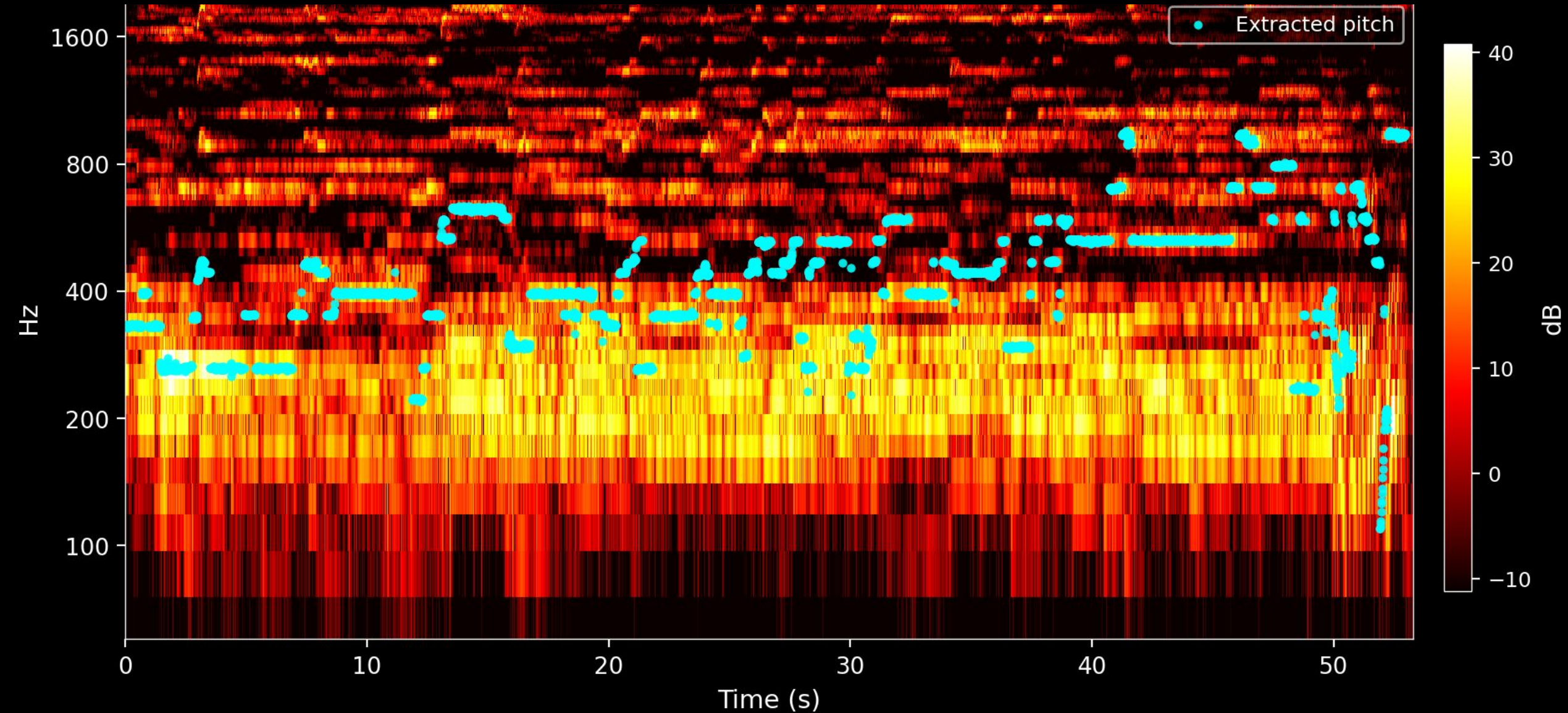
Spectrogram (RAW) Input 4



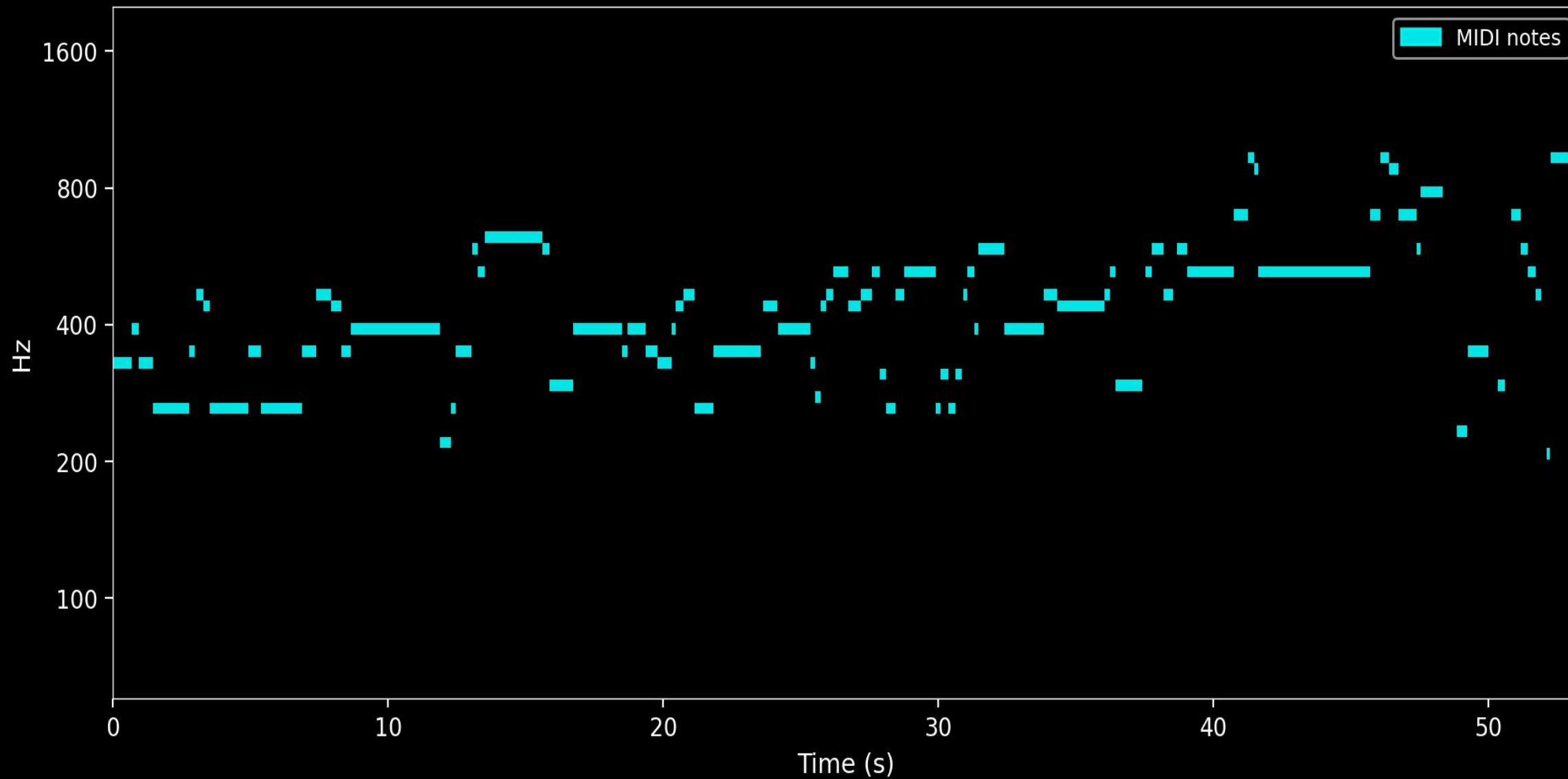
Spectrogram (HPSS) Input 4



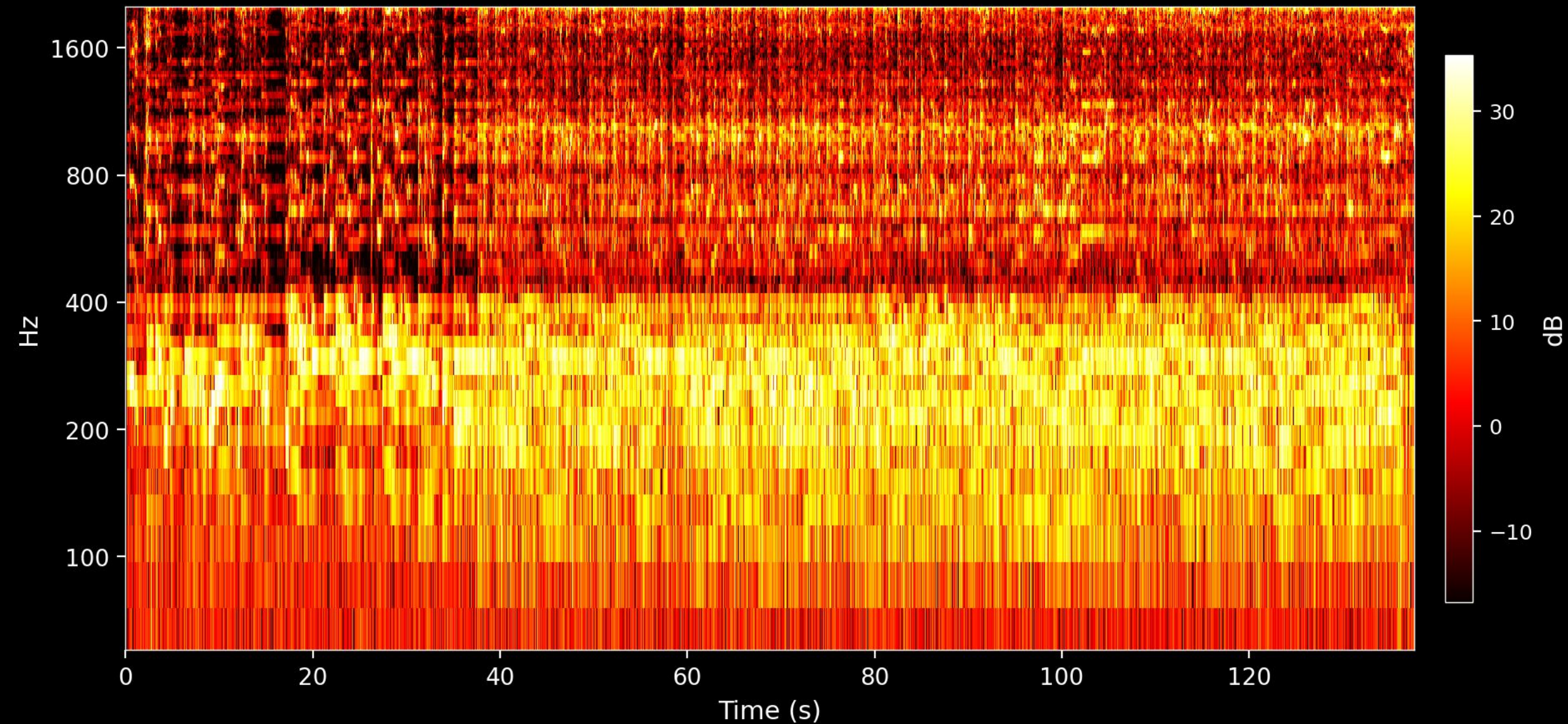
Spectrogram (Pitch Contour from Viterbi) Input 4



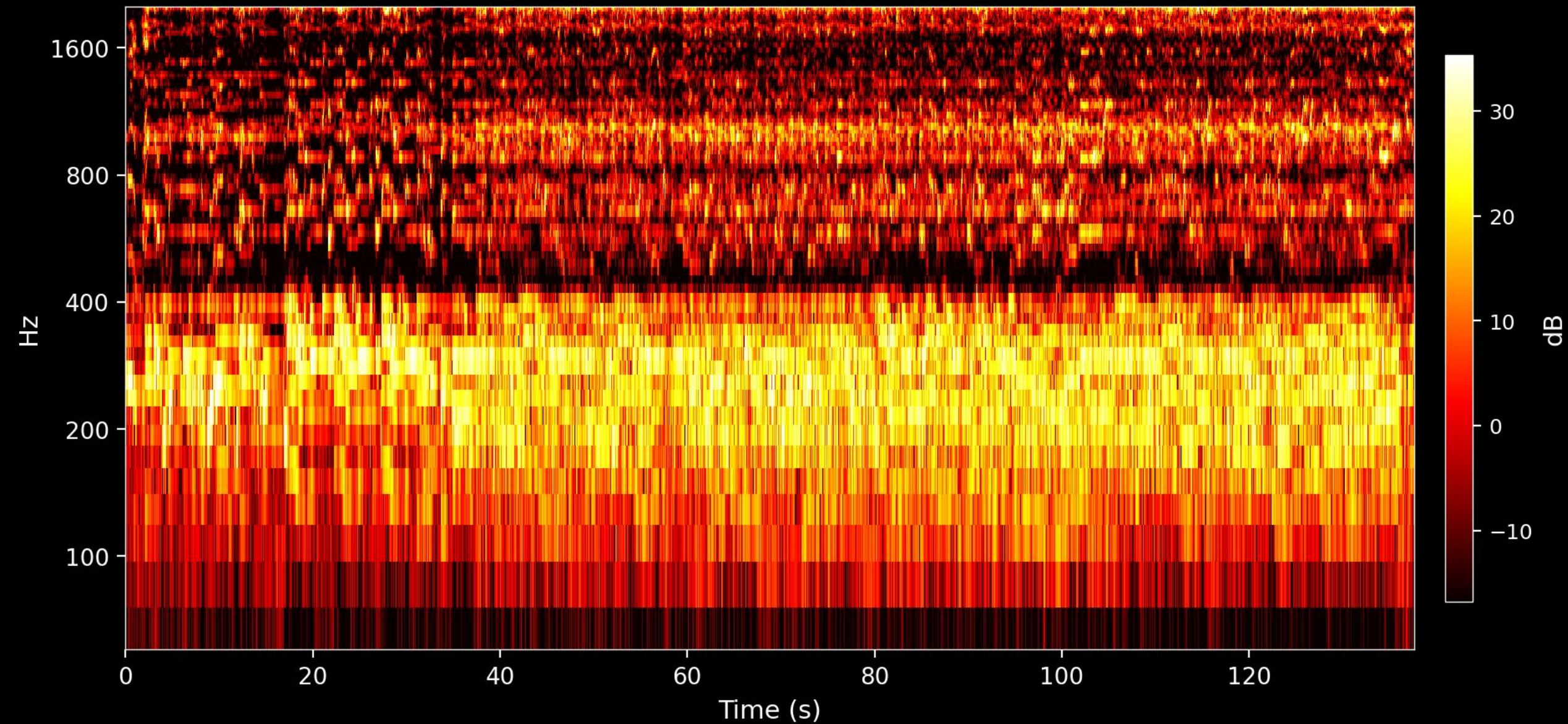
Spectrogram (MIDI Output) Input 4



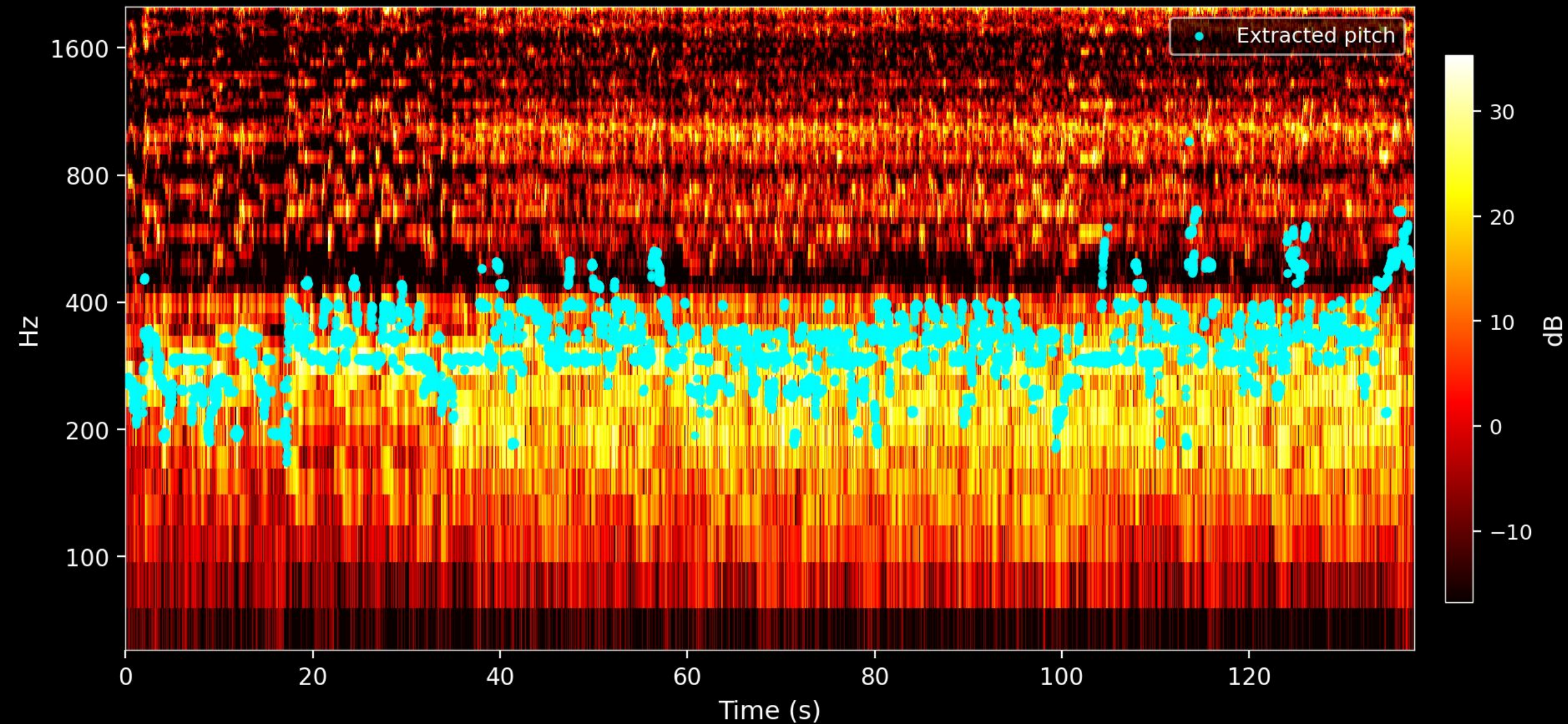
Spectrogram (RAW) Input 3



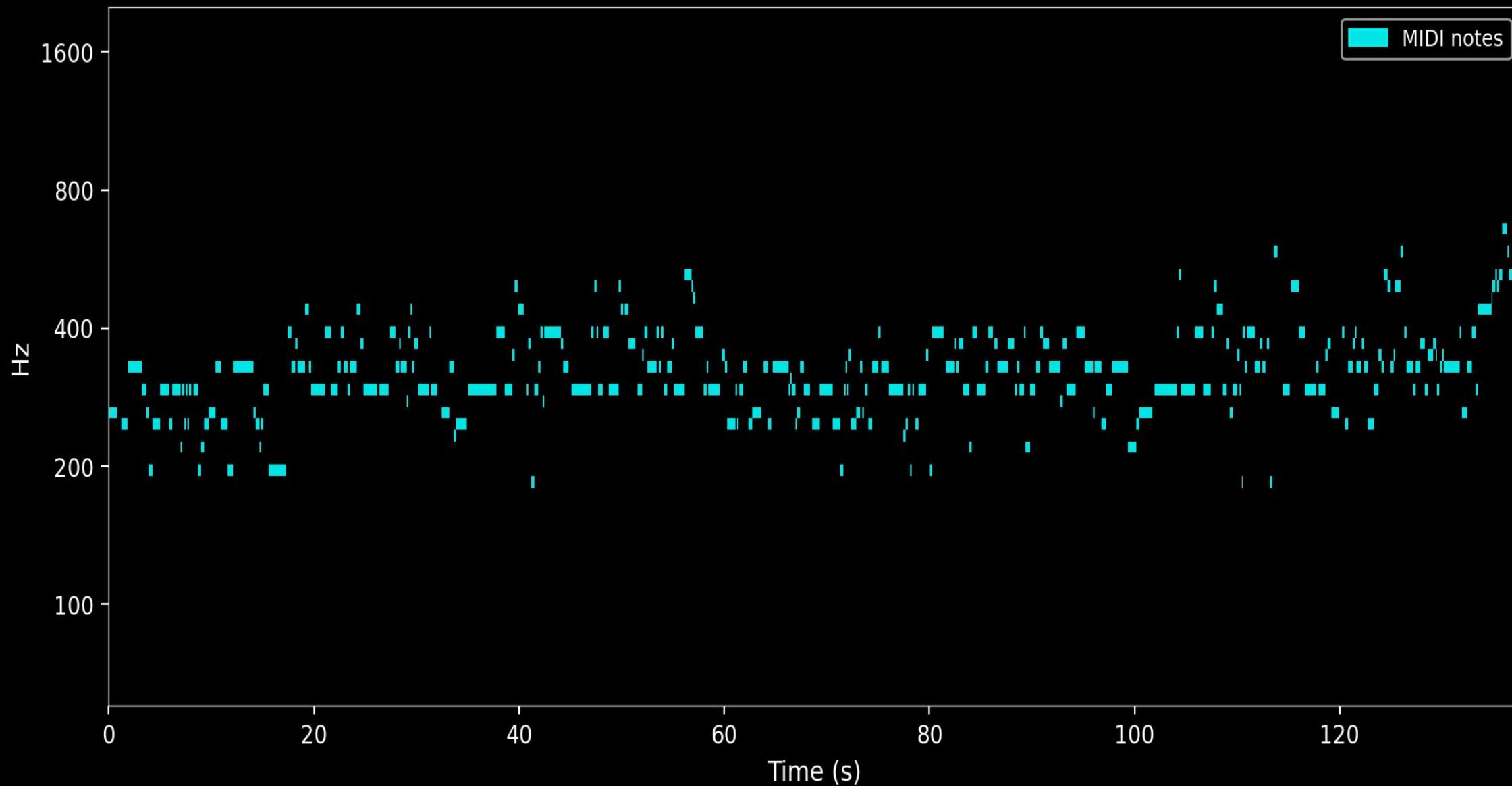
Spectrogram (HPSS) Input 3



Spectrogram (Pitch Contour from Viterbi) Input 3



Spectrogram (MIDI) Input 3



Accuracy Evaluation (monophonic)

Evaluation Method: Compared Digital Ear's melody-only MIDI output against Spotify's Basic Pitch (SOTA polyphonic AMT). Note-level matching with $\pm 0.3s$ onset tolerance.

Three pitch criteria: exact match, ± 1 semitone, and octave-tolerant (± 12 semitones).

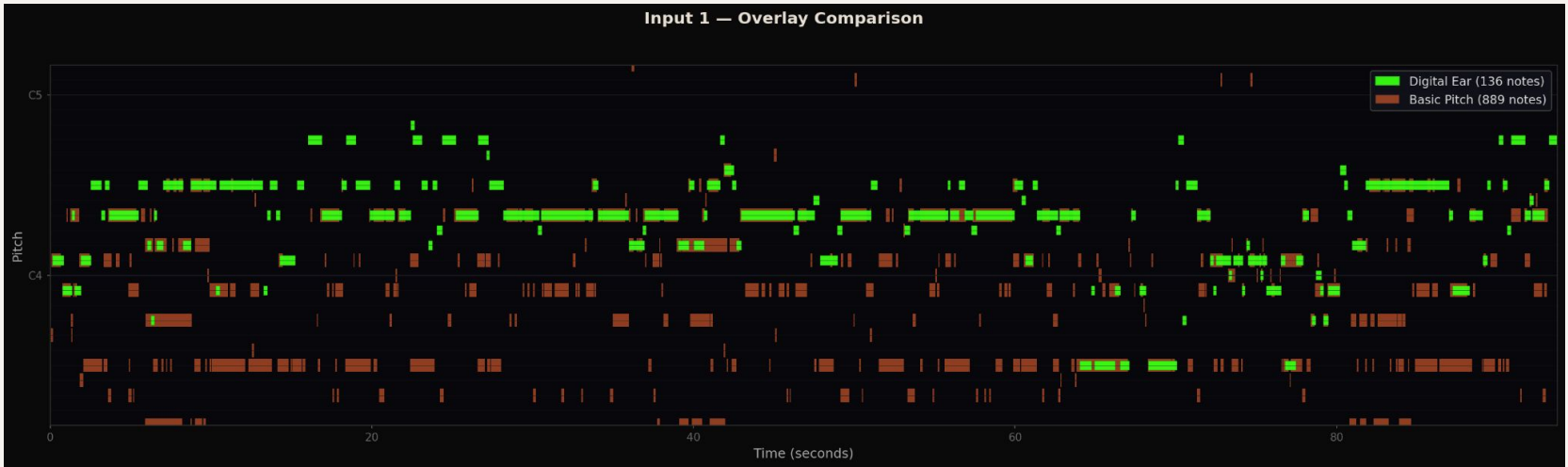
Precision measures "of the notes DE found, how many were real?"

Recall is low by design as DE extracts only the melody line, while BP captures all instruments.

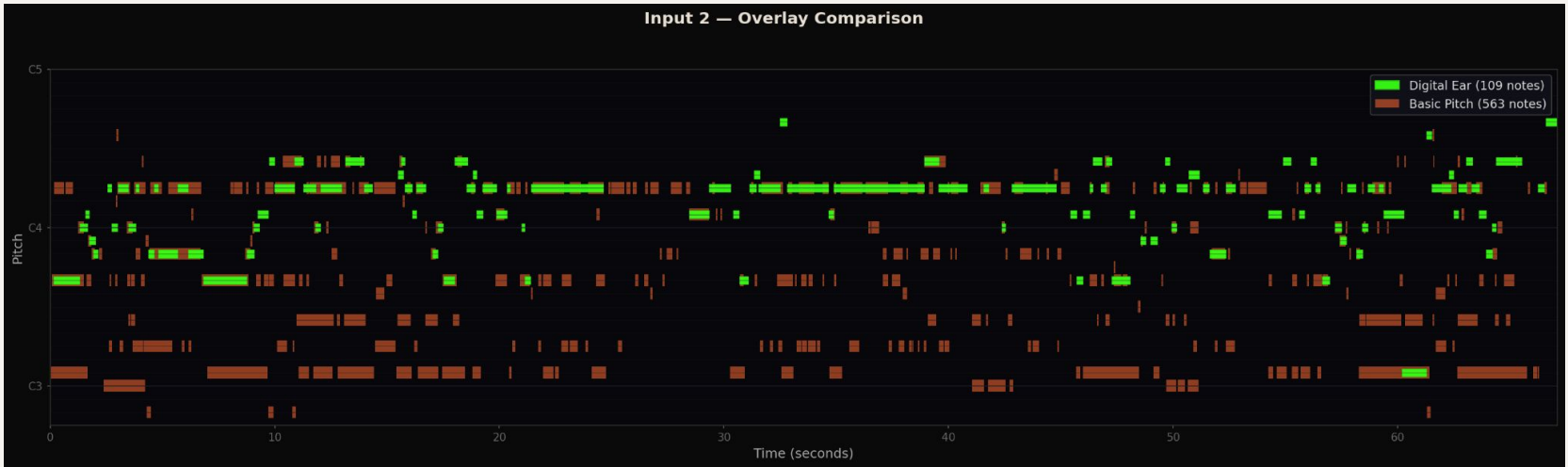
Input	DE Notes	BP Notes	Strict Precision	Oct-Tolerant Precision	Oct F1
1	136	889	56.6%	92.6%	24.6%
2	109	563	67.0%	97.2%	31.5%
3	245	1414	74.3%	98.8%	15.1%
4	82	936	82.9%	93.9%	15.1%
Avg			70.2%	95.6%	25.1%

Key takeaway: Digital Ear achieves 95.6% average octave-tolerant precision. Nearly every note it finds is a real note in the music.

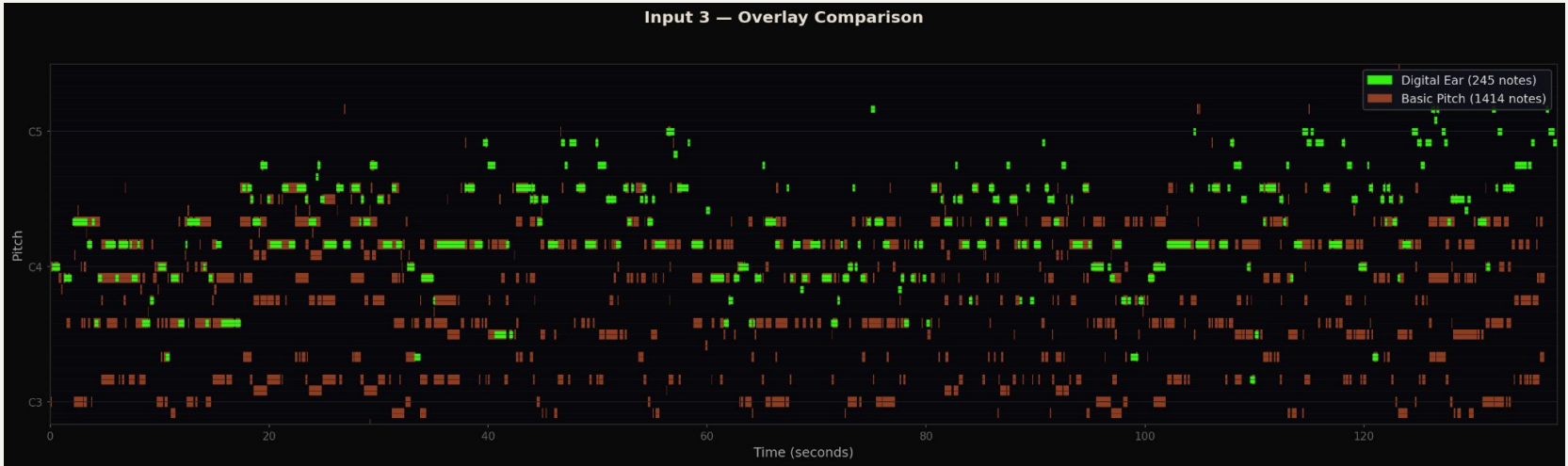
Input 1



Input 2



Input 3



Input 4

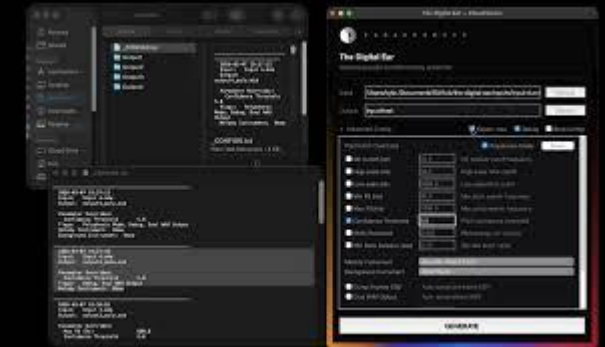
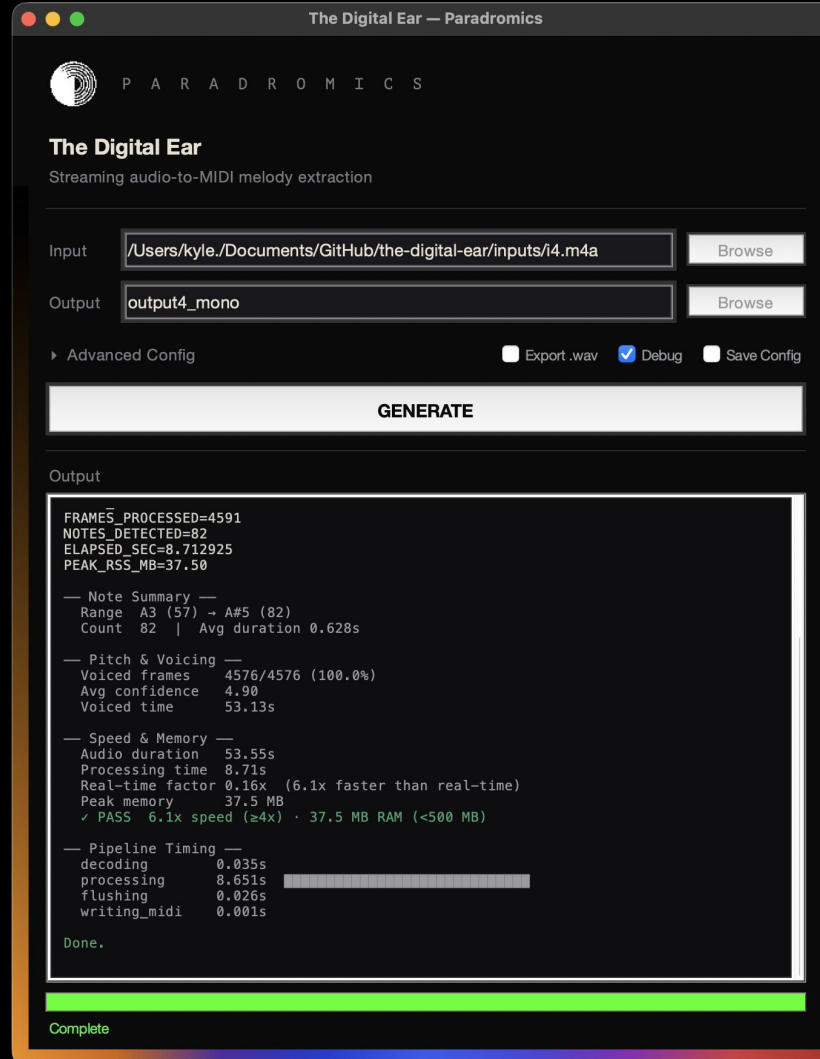
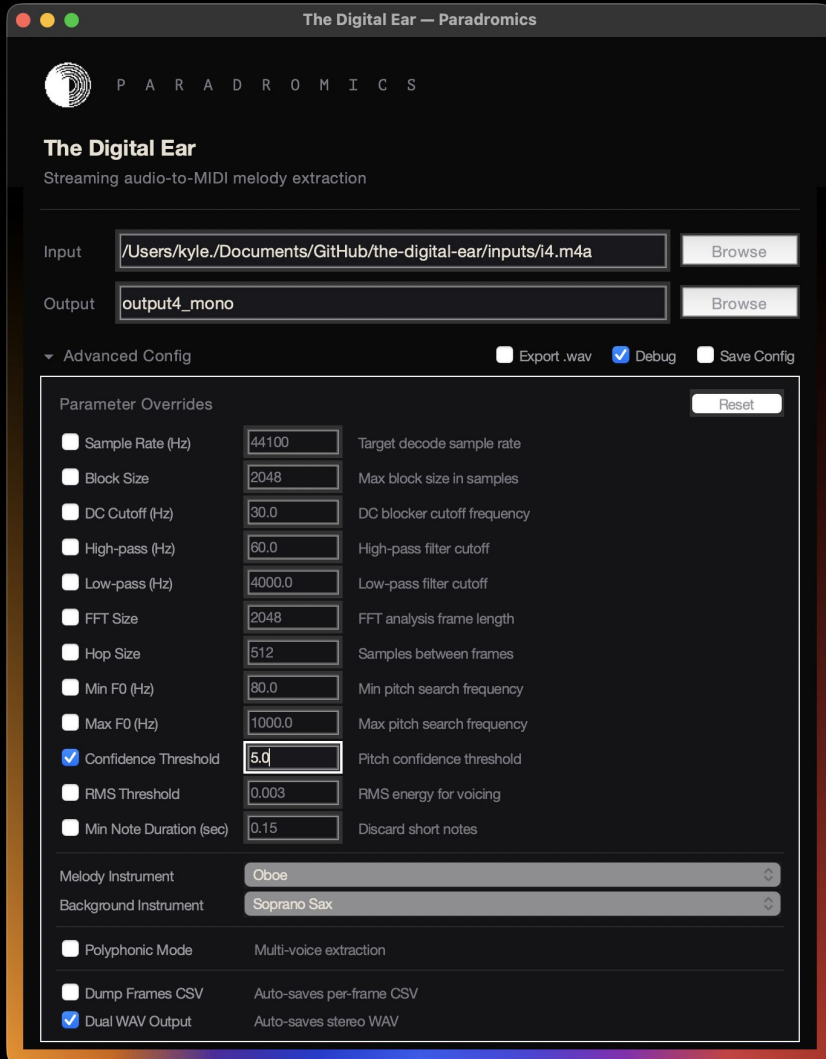


Extras

Note: These extras are for ease of development, tuning, and export, and are therefore not considered as part of the project solution for performance reasons.

Additional Export Options:

- MIDI -> .wav export
- Dual Stereo overlay export
- Polyphonic output
- CSV frame dump (for debugging)



gui.py icon in taskbar

Live Musicbox

Note: These extras are for ease of development, tuning, and export, and are therefore not considered as part of the project solution for performance reasons.



Resources

Fitzgerald, D. (2010) Harmonic/Percussive Separation using Median Filtering. 13th International Conference on Digital Audio Effects (DAFX10), Graz, Austria, 2010

Salamon, J. and Gómez, E., Melody Extraction from Polyphonic Music Signals using Pitch Contour Characteristics, IEEE Transactions on Audio, Speech, and Language Processing, Vol. 20, No. 6, pp. 1759–1770, 2012.

Bittner, R. M., Salamon, J., Essid, S., and Bello, J. P., Melody Extraction by Contour Classification, Proc. International Society for Music Information Retrieval Conference (ISMIR), 2015.

<https://www.justinsalamon.com/melody-extraction.html>